Beyond Scripts

Extending Azure SRE Agent with PowerShell Automation

Jan Egil Ring

Microsoft Cloud Solution Architect

PSConf EU 2026

📋 Agenda

1. Introduction & Problem Framing
2. SRE Agent Fundamentals + Quick Start Demo
3. Demo 1 — Automation Runbooks via MCP
4. Demo 2 — Logic Apps + Inline PowerShell
5. Demo 3 — PowerShell Universal MCP
6. Demo 4 — Custom Incident Response + Drift Detection
7. Demo 5 — Intelligent Alert Triage
8. Best Practices & Wrap-up + Q&A

😮‍💨 The Challenge: Operational Toil

🚨 Incident Response

Manual investigation, log digging, ad-hoc remediation — at 3 AM

📊 Alert Triage

Hundreds of alerts, many false positives, context switching between tools

🔄 Drift Detection

Infrastructure slowly diverges from desired state — discovered too late

⏱️ These repetitive tasks consume the time we should spend on strategic initiatives

The Evolution of Automation

Scripts
Manual trigger
Single task

→

Scheduled
Cron / Task Scheduler
Runbooks

→

Event-Driven
Webhooks / Alerts
Logic Apps

→

AI Agents
Autonomous reasoning
Contextual decisions

Each step adds intelligence — agents add reasoning

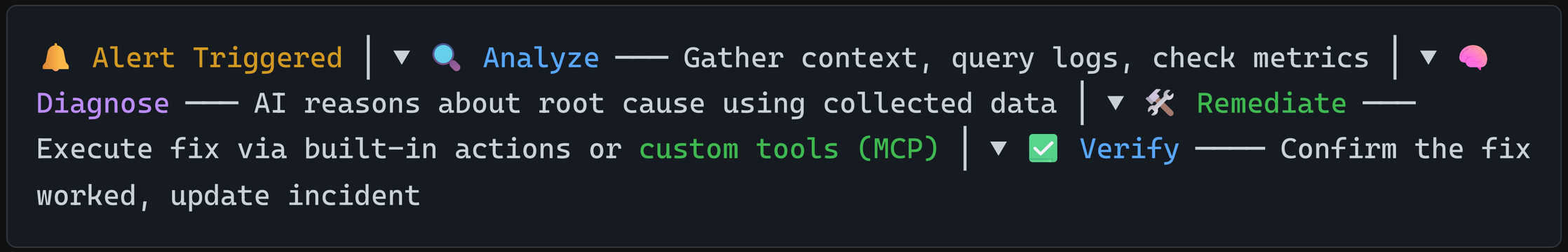
🤖 What is Azure SRE Agent?

Microsoft's AI-driven platform for Site Reliability Engineering

- ✅ Built-in Azure knowledge & diagnostics
- ✅ Custom sub-agent extensibility
- ✅ Model Context Protocol (MCP) for external integrations
- ✅ Integrates with Azure Monitor, ServiceNow, GitHub, and more

⚙️ How SRE Agent Works

The investigation and remediation flow



Two modes: Always-on continuous monitoring Active on-demand investigation

🔌 Model Context Protocol (MCP)

The universal connector for AI agents

What is MCP?

- Open protocol for AI ↔ tool communication
- Standardized interface — "USB-C for AI agents"
- Describes available tools & their capabilities
- Handles invocation & response formatting

Why it matters for us

- Expose PowerShell code as an agent tool
- Agent discovers what your code can do
- No custom SDKs or deep AI knowledge needed
- Reuse existing scripts & runbooks

```
SRE Agent ⟶MCP⟶ MCP Server ⟶ Your PowerShell Code (tool descriptions, invocations, results)
```

🔧 PowerShell Integration Patterns

Three approaches to connect your PowerShell with SRE Agent

1 Azure Automation

Runbooks via MCP connector

- Managed identity auth
- Complex orchestration
- Built-in logging
- Hybrid Worker support

Best for: Enterprise scenarios

2 Logic Apps + PS

Inline PowerShell in workflows

- Visual workflow designer
- 200+ connectors
- Inline PS execution
- HTTP trigger for MCP

Best for: Integration scenarios

3 PowerShell Universal

PS scripts as REST APIs

- Full PS runtime
- Custom API endpoints
- Dashboard & reporting
- Script-first approach

Best for: PS-heavy teams

Demo Environment

Azure infrastructure supporting the five demos

Managed identities + least-privilege RBAC across all integrations | Azure Files mount for PSU Repository

🎬 Demo 1: Azure Automation Runbooks via MCP

Scenario: SRE Agent triggers a PowerShell runbook to scale an App Service Plan

What you'll see:

- SRE Agent receives alert and begins investigation
- Agent invokes Logic Apps Standard MCP server
- Logic App workflow triggers Azure Automation runbook
- Runbook scales App Service Plan with managed identity

🎬 Demo 2: Logic Apps with Inline PowerShell

Scenario: Incident enrichment with inline PowerShell execution

What you'll see:

- Two patterns: Azure Automation connector + inline PowerShell
- SetAppServicePlanSku workflow calls Automation runbooks
- EnrichIncident workflow runs inline PowerShell code
- Enriched incidents posted to Teams with full context

🎬 Demo 3: PowerShell Universal API

Scenario: Backend fix script via PowerShell Universal MCP

What you'll see:

- PSU exposes PowerShell scripts as REST API endpoints
- SRE Agent calls Fix Backend Server endpoint
- PowerShell script executes backend remediation
- Agent receives execution status and results

🎬 Demo 4a: Custom Incident Response + Auto-Scale

Scenario: Custom agent investigates an alert, defensively scales the App Service Plan on capacity signals, and posts an audited summary to Teams

What you'll see:

- `/api/slow` fires a Sev2 response-time alert → agent scales one tier (P0v3 → P1v3)
- `/api/error` fires a Sev1 5xx alert → identified as false positive, **not** scaled
- Defensive scale-up is capped at P2v3 — beyond that needs human approval
- Every incident posts a structured Teams summary with the actual "Remediation Taken"

🎬 Demo 4b: Infrastructure Drift Detection

Scenario: Detect and remediate storage governance drift via PowerShell Universal MCP

What you'll see:

- Agent calls PowerShell Universal MCP drift detection tool
- Detects storage governance drift (access settings changed)
- Executes PowerShell remediation script to restore settings
- Restarts Logic App to verify connectivity restored

🎬 Demo 5: Intelligent Alert Triage

Scenario: Azure Monitor alert → AI root cause analysis → PowerShell fix

What you'll see:

- Multiple Azure Monitor alerts routed to SRE Agent
- AI correlates alerts and identifies root cause
- Agent selects appropriate PowerShell remediation
- End-to-end: alert fires → problem fixed → incident closed

🤔 When to Use What?

Decision matrix for your automation strategy

Scenario	Traditional PS	SRE Agent	Hybrid
Simple scheduled task	✅ Best fit	Overkill	—
Known remediation, fixed input	✅ Best fit	Possible	—
Alert requiring investigation	Limited	✅ Best fit	Good
Multi-signal root cause analysis	❌ Hard	✅ Best fit	—
Compliance + remediation	Check only	Good	✅ Best fit
Complex orchestration	Possible	Good	✅ Best fit

💡 **Rule of thumb:** If it needs *reasoning*, use an agent. If it's *deterministic*, use a script.
For most real-world scenarios, hybrid wins.

✅ Best Practices

🔒 Security

- Managed identities — no stored credentials
- Least-privilege RBAC for all integrations
- Key Vault for any secrets
- Audit logging on all agent actions

🔗 Hybrid Approach

- Agent for reasoning, PowerShell for execution
- Reuse existing runbooks — don't rewrite
- Start small: one scenario, prove value
- Build a library of MCP-connected tools

💰 Cost Optimization

- Always-on vs Active mode — choose wisely
- Scope agent monitoring to critical resources
- Use traditional PS for simple, predictable tasks

🧪 Testing & Reliability

- Fallback to manual PS if agent is unavailable
- Test MCP integrations independently
- Monitor agent decision quality over time

📚 Resources & Links

🔗 Documentation

- Azure SRE Agent Documentation
- Model Context Protocol Spec
- Azure Automation Docs
- PowerShell Universal Docs

🐙 This Session

All demos, code & slides:

github.com/janegilring/Presentations

Thank You

Back to main deck...